

Simplifyr

System Architecture

Simplify the signal. Preserve the truth.

A practical overview of how Simplifyr processes logs, handles changes, uses AI, and is being prepared for larger deployments.

1. What Simplifyr Does

Simplifyr is a log processing and normalization platform. The main problem we are trying to solve is that logs from different devices and applications do not follow one common format.

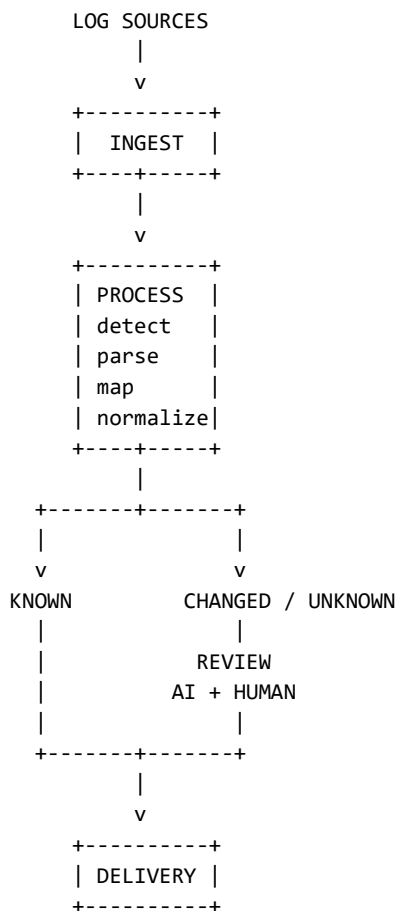
A firewall might send Syslog, an application might send JSON, and an older security product might use CEF or LEEF. Even when two systems contain the same information, they may use different field names for it.

```
src_ip      source_ip      sourceAddress      client_ip
  \          |            |            /
    all can represent source.ip
```

Instead of writing separate logic for every downstream system, Simplifyr provides a common normalization layer. It also keeps the original event so that normalization does not mean losing data.

2. Overall Architecture

The application is split into a few clear stages. Each stage has a specific job, which also makes it easier to replace or scale parts of the system later.



3. Ingestion

The first step is getting the event into Simplifyr. Events can arrive individually or as part of a batch. At ingestion time, we keep information such as the event ID, timestamp, source and original payload.

Keeping the original payload at this point is important. If parsing or mapping changes later, we still have the data that was actually received.

4. Parsing and Format Detection

Once an event arrives, Simplifyr determines which parser should handle it. The current architecture accounts for formats such as Syslog, JSON, XML, CSV, CEF, LEEF and raw text.

The parser extracts the fields it can understand. If a part of the event cannot be parsed, the original information is not simply discarded.

5. Normalization and Mapping

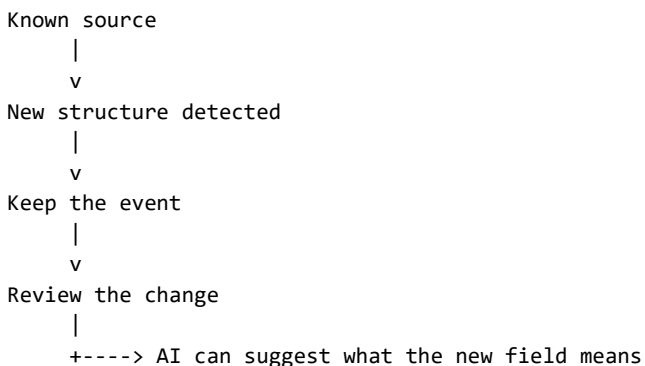
Different sources often use different field names for the same concept. Simplifyr uses mappings to connect source-specific fields to a common structure.

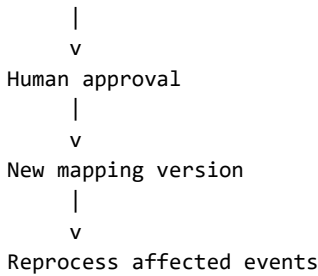
Example source field	Normalized field
src_ip	source.ip
dst_ip	destination.ip
action	network.action
rule_id	rule.id
severity	threat.level

Mappings are versioned. This gives us a way to know which mapping was used for an event and allows future changes to be handled without losing the history of previous processing.

6. What Happens When a Log Changes?

One of the more important cases is when a source changes its log structure. This could happen after a vendor update, a configuration change, or a new field being introduced.





The reason for doing this instead of automatically guessing is that a wrong mapping can make security data misleading. We want changes to be visible rather than silently changing the meaning of an event.

7. AI in Simplifyr

AI is used to assist with parts of the system where interpretation is useful. It is not the only thing deciding what a log means.

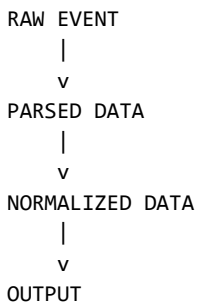
- Suggesting mappings for unfamiliar fields
- Helping analyse new or changed log structures
- Assisting the review process
- Using previously approved information to improve future suggestions

The regular processing path is still deterministic. When the system already knows how to parse and map something, there is no need to ask a model to make the decision again.

In other words, AI is there to reduce manual work, not to replace the parts of the pipeline that can be handled reliably with normal application logic.

8. Preserving Data and Tracking What Happened

A normalized event is useful, but the original event is still important. Simplifyr keeps the different stages of processing so that we can trace how an output was produced.



This helps with debugging, investigating unexpected results, reviewing mappings and understanding which version of a mapping was used.

9. Failed and Unknown Events

Not every event will be successfully understood. These cases are kept separate from successfully normalized data.

Case	Handling
Normal event	Parse, normalize and deliver
Changed or unknown structure	Preserve the event and send it through the review flow
Processing failure	Keep it separate for failure or dead-letter handling

This makes it easier to see which events need attention instead of mixing failures with valid output.

10. Current Architecture and Scaling

The current version is designed to run on a local machine and still provide the complete workflow. At the same time, the services around the processing logic give us a path to larger deployments.

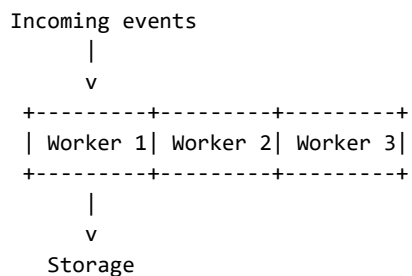
We are treating scaling as an incremental process. We do not need to start with a large distributed cluster just to run the application. If the workload increases, individual parts can be expanded.

Need	How we plan to handle it
More event processing	Add more processing workers
More concurrent users	Run multiple API instances
Higher event volume	Use Kafka to distribute events
More raw data	Use object storage such as MinIO
Larger AI workloads	Use dedicated model-serving resources
Larger deployments	Use multiple service instances and orchestration

11. Scaling Path

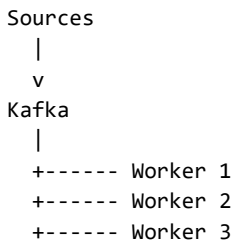
11.1 More workers

If processing becomes the bottleneck, the same processing work can be divided among multiple workers.



11.2 Kafka for higher event volume

Kafka can be used as the event transport layer when the number of incoming events becomes large enough that producers and consumers need to be separated.



11.3 Object storage

As raw event storage grows, object storage can take more of the long-term archive workload while the main database focuses on application and operational data.

11.4 Distributed deployment

For larger environments, services can be deployed as multiple instances and managed using an orchestration platform such as Kubernetes.

12. What Is Implemented Today

The current implementation is focused on proving the complete path through the application, rather than building individual components in isolation.

- Source onboarding
- Log ingestion and batch processing
- Multiple log formats
- Parsing and field mapping
- Normalization
- Versioned mappings
- Drift detection
- Quarantine and review
- AI-assisted analysis
- Human approval
- Event reprocessing
- Raw-event preservation and provenance
- Live event monitoring
- Analytics and search
- Output profiles and export
- Configurable destinations
- Containerized deployment

13. Future Plans

The current system gives us the foundation. The next improvements are mainly about making the same workflow more scalable, easier to operate and more useful with larger amounts of data.

Area	Planned direction
Distributed processing	Move more processing into independently scalable workers.
Streaming	Use Kafka more extensively for continuous high-volume streams.
Storage	Improve object-storage support and retention for large raw-event archives.
AI assistance	Improve mapping and drift suggestions using better models and approved knowledge.
Access control	Add stronger authentication and role-based access control for multi-user deployments.
Observability	Expand monitoring of processing speed, failures, queues and resource usage.
Onboarding	Make onboarding faster while keeping important mapping decisions reviewable.

14. Technology Stack

The stack is based on technologies that are practical for the current project and have clear paths to larger deployments.

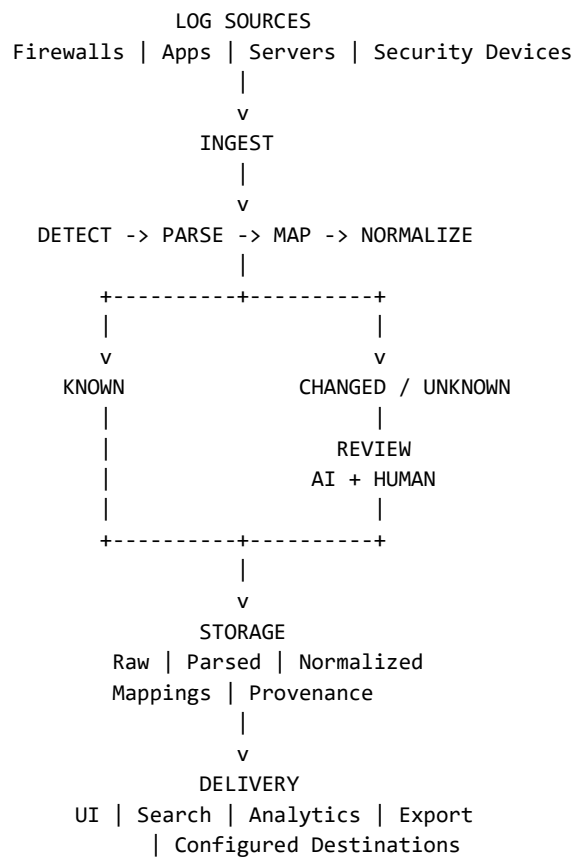
Part	Technologies	Used for
Frontend	React, TypeScript, Vite, Tailwind CSS	Web application, dashboards, configuration and live views
Backend	Python, FastAPI, SQLAlchemy	API and core application processing
Database	PostgreSQL, SQLite	Application data, events, mappings and configuration
AI	Ollama, Groq, Google Gemini	Mapping assistance and analysis
Infrastructure	Docker, Docker Compose, Kafka, Redis, MinIO	Containerized deployment and scalable supporting services

15. Why We Built It This Way

The main architectural decisions came from the problems we wanted to solve rather than from trying to use as many technologies as possible.

Decision	Why
Keep the original event	We should be able to go back to what was actually received.
Version mappings	We need to know how an event was interpreted at a particular point in time.
Do not silently guess	A wrong mapping can be worse than an event that needs review.
Use AI where it helps	It can reduce manual work without making the whole pipeline dependent on it.
Separate processing from infrastructure	The same processing logic can be used as the deployment grows.

16. Final View



Simplifyr is built around a straightforward idea: different log sources can remain different at the input, while downstream systems get a consistent representation. At the same time, the original data is preserved and changes can be reviewed instead of being silently accepted.